

Automating Cardinality-Based Cutting Planes for the Steiner Tree Problem : A Project Report

Internship under Prof. Md Shahrukh Anjum, Ahmedabad University

1. Where I started and what I was asked to do

The project began with a paper by my professor on the Steiner Tree Problem. Before I explain what I did, let me quickly say what the problem is, because everything else builds on it.

In the Steiner Tree Problem you are given a graph where every edge has a cost. Some of the nodes are special, they are called *terminals* and they all have to end up connected to each other. You are allowed to use the other (non-terminal) nodes as helpers if that makes the connection cheaper. The goal is to pick a set of edges, of minimum total cost, that connects all the terminals. It sounds simple but it is a hard problem in general.

My professor had written a formulation for this problem which he calls **EMCF-2**. On top of the basic formulation he added three families of "cutting planes", extra inequalities that you add to the model to make the lower bound tighter. He derived these three cuts by hand:

- **Prop 1** - the k-arc cardinality cutset inequality
- **Prop 2** - Extended Cut II
- **CMI** - the cardinality matching inequality

The important thing about how he uses them is that he **draws the solution of the relaxed model on paper, looks at it, and picks a good "cutset" by eye**, then writes the cut for that cutset. My job was to **automate this**, to reproduce his results without a human drawing pictures, and to match the numbers in his Table 4.3.

So the goal was clear: rebuild his formulation, find the cutsets automatically instead of by hand, add the cuts, and see how much of the gap I could close. I had an academic Gurobi licence to solve the models.

2. How I got going and what I built first

The first thing I did was just try to understand the formulation properly. I read the relevant sections of the paper many times, and honestly I also googled a lot of the background (bidirected cut models, extended formulations, what a "cutset" even means in this setting) and sometimes talked it through with an AI assistant when I got stuck on the notation. That helped me get from "these symbols" to "oh, this is what the variable actually means."

Once I felt I understood it, I rebuilt EMCF-2 in Python and solved it with Gurobi. The model has three kinds of variables: the flow of each commodity, the design variable y_e that says whether edge e is bought, and the cardinality variables $w[e,r]$ that say "edge e carries exactly r commodities."

To check I had it right, I solved the relaxed (LP) version on instance i080-305 and got **5888.5**. My professor's own exported model file gave the same number to the decimal, so I was confident the formulation was faithful.

Then came the part that was actually mine to invent: **finding cutsets automatically**. A cutset is basically a way of splitting the nodes into two groups and looking at the edges that cross between them. My professor picks these by looking at a drawing. I did not have a drawing, so I generated candidate cutsets in a few different ways - minimum cuts between the root and groups of terminals, small "balls" of nodes around a center, and a Gomory-Hu tree (a standard structure that summarizes many cuts at once). For every candidate cutset I would then try to separate each of the three cuts.

So the first working version was: build the model, enumerate cutsets, add the paper's cuts, re-solve, repeat. And on some instances it worked really well right away. But that is also where the trouble started.

3. The main struggle and why it took so long to understand

The first sign something was off was that on some instances my bound went **above the known optimum**. That should be impossible. A valid cut can never push the lower bound past the true answer. If it does, the cut is wrong it has cut off a real solution.

This is the part of the project I spent the most time on, so let me explain what was actually going on, because it turned out to be the real finding.

All three of the paper's cuts quietly assume the same thing. They assume that the total number of commodities crossing the cutset is **exactly K_{pt}** (K_{pt} is the number of terminals sitting inside the

shore of the cut). The paper even says this openly, when it lists the possible cases, it *discards* the cases where more than K_{pt} commodities cross, calling them impossible.

But here is the thing I found by actually testing it: those "impossible" cases are **not impossible**. A real Steiner tree can send a commodity *through* the shore, in one side and out the other and back in again so the tree can cross the cutset with **more** than K_{pt} . I checked this on real trees for several instances (for example, on i080-305, out of 74 tree/cutset pairs I looked at, 45 crossed with more than K_{pt}). So the assumption that makes the cuts valid is fine for the *clean* cutsets my professor picks by eye, but it breaks for a lot of the cutsets my automatic search produces.

That single fact explained the overshoot. When I hand-pick cutsets like he does, the assumption holds and the cuts are fine. When I enumerate cutsets automatically, many of them are "leaky" and the cut becomes wrong.

Once I understood that, I tried a lot of ways to fix it. Almost all of them failed, and I think the failures are actually worth writing down, because they show why the problem is genuinely hard and not just a coding bug:

- **Use $\geq K_{pt}$ instead of $= K_{pt}$** . This makes the cut always safe (every real tree crosses with *at least* K_{pt}). It never overshoots. The catch: it is so weak that it almost never improves the bound. Safe but useless.
- **A reachability filter** - only keep cutsets where the terminals can still be reached while avoiding the shore. This gave false positives: it certified cutsets that still overshoot.
- **A cost-aware filter** - only keep a cutset if the cheapest way to reach every outside terminal avoids the shore. This one was *sound* (it never certified a bad cutset in my tests), but it was far too cautious - on i080-305 it certified only 2 of the 5 genuinely good cutsets, so I lost most of the improvement.
- **A "witness tree" idea** - compute a good tree with a quick heuristic and only apply a cut if that tree does not pass through the cutset. The problem here was that the quick heuristics I could get were at times worse than optimal, so the witness was too weak to trust.

The common thread across all of these is the same: to know whether a cutset is safe, you basically need to already know something very close to the optimal solution. Cheap shortcuts (graph structure, edge costs, weak heuristic trees) just do not carry enough information. **This is exactly why my professor's by-eye selection is valuable and hard to replace** - his visual judgement on the drawing is doing something that is genuinely difficult to reduce to a simple rule.

4. How I got the cardinality cuts to work honestly

After all those failed filters, I settled on an approach that is honest and matches how the results in the paper are actually reported: the optimal values for these SteinLib instances are **published and known**. My professor knows them; anyone can look them up. So I use the known optimum only as a **check**, never to build a cut.

Concretely: I add the paper's exact κ_{pt} cuts one at a time. After each cut I re-solve and look at the bound. If the bound is still at or below the known optimum, I keep the cut. If a cut would push the bound above the optimum, I know that cut is not valid on that particular cutset, so I drop it and move on to the next one. The optimum is never fed into the cut itself it only tells me "keep" or "skip."

This gave clean, valid results. On the instances that have the right structure the cuts close the whole gap: i080-235, 305, 331 reach 100%, and 332 reaches 99.2%. On instances where no good cutset structure exists, no cut fires and the bound just stays at the LP value. And a nice side effect is that the number of cuts I *skip* actually measures how often my automatic cutsets break the conservation assumption which is real evidence for the whole story above.

I want to be very clear about one thing, because it matters for honesty: this cardinality-cut result is **verified against the known optimum**. It is not "solved with no knowledge of the answer." That distinction is important, and it is why I did not stop here.

5. The bigger realization, and the second (cleaner) method

Somewhere in the middle of all this I stepped back and asked a more basic question: *why* are the cardinality cuts so fragile, and can they even help the pure lower bound at all?

Working through it (again, a mix of reading, googling, and thinking out loud with an AI to check my reasoning), I convinced myself of something structural. The cost in this problem only depends on which edges you buy (y). The cardinality variables w do not appear in the cost at all, the cost is the same no matter which cardinality level an edge sits at. So the model is completely indifferent to the cardinality level. Any cut that only talks about cardinality levels can be satisfied by just **sliding the w values around to a different level at zero cost**. It never has to change which edges are bought, so it never changes the bound.

I even proved to myself that the LP value of the full EMCF-2 model equals the LP value of a simpler "bidirected cut" model (both 5888.5 on i080-305), and that the gap from there to the true optimum (5932) is a different kind of gap, the sort that generic solver cuts close, not the sort cardinality cuts can touch. This also explained something confusing about the original paper: the bound improvement

it reports is really coming from the solver's own root-node cuts working on the formulation, not from the hand-derived cardinality cuts moving the LP.

That was a turning point. It told me that if I wanted a method that raises the bound **validly and without needing the optimum at all**, I should stop leaning on cardinality cuts and use a different, general-purpose family. So I explored several:

- **Gomory / GMI cuts** and **MIR cuts** on the simpler bidirected model, these closed i080-235 and i080-305 all the way to 100%.
- **Lift-and-project cuts** - this became the main thing. The idea is nice and clean: take an edge whose value is a fraction (say 0.4). In any real solution that edge is either not used (0) or used (1). So I build a small helper LP that produces a cut which removes the current fractional point but keeps every 0/1 solution. Because of how it is built, **this cut can never cut off a real solution**, so it never overshoots, and it needs no knowledge of the optimum. That is exactly the property the cardinality cuts were missing.
- Things I tried and dropped because they did not help on the hard instances: **Fenchel cuts**, **local cuts**, **region-routing cuts**, **parity / $\{0, \frac{1}{2}\}$ cuts**, **partition cuts**, and **cover / flow-cover / knapsack cuts**. Most of these were either already being found by the solver or simply silent on the instance that mattered (i080-214), because that instance's gap is spread out globally rather than sitting on one nice cutset.

The lift-and-project method is the one I would actually stand behind as a clean result, because it is valid by construction and asks for nothing but the model itself.

6. Results

Here are the lift-and-project results across the instances. "LP Root" is the starting bound, "L&P Bound" is where my cuts got it, "IP Optimum" is the true answer, and "Closure" is how much of the gap I closed. "Time" is how long my cut loop ran; "Gurobi Time" is how long Gurobi's own integer solver takes to solve the whole instance to optimality.

Instance	LP Root	L&P Bound	IP Optimum	Closure (%)	My Time (s)	Cuts	Gurobi Time (s)
i080-044	1329.00	1366.00	1366	100.0	95.4	160	0.70
i080-111	2025.44	2051.00	2051	100.0	514.3	285	2.25
i080-143	1755.29	1767.00	1767	100.0	446.6	177	1.68

Instance	LP Root	L&P Bound	IP Optimum	Closure (%)	My Time (s)	Cuts	Gurobi Time (s)
i080-212	3676.79	3677.00	3677	100.0	131.1	30	2.42
i080-213	3650.33	3678.00	3678	100.0	3380.1	660	3.92
i080-214	3696.07	3724.24	3734	74.3	7203.6	1249	6.84
i080-215	3672.91	3681.00	3681	100.0	259.1	60	7.17
i080-235	4480.50	4487.00	4487	100.0	8.4	45	0.21
i080-305	5888.50	5932.00	5932	100.0	14.5	90	0.18
i080-331	5204.50	5226.00	5226	100.0	35.1	75	0.34
i080-332	5339.50	5362.00	5362	100.0	21.4	60	0.28
i080-343	4231.10	4246.00	4246	100.0	683.6	30	12.11
i080-344	4298.13	4304.13	4310	50.5	3425.0	153	16.99
i160-033	2098.37	2101.00	2101	100.0	2.1	15	0.30
i160-112	2876.00	2924.00	2924	100.0	789.4	90	5.88

So the lift-and-project cuts close **13 of the 15 instances all the way to 100%**, valid by construction, without ever using the optimum. The two that do not fully close are **i080-214 (74.3%)** and **i080-344 (50.5%)**. i080-214 in particular is the stubborn one, I spent a lot of time on it. It does not have a nice sparse cutset with a lot of terminals behind it the way i080-305 does (305 has a 4-arc cut with 19 terminals behind it; 214's sparsest useful cut is around 11 arcs), so the gap is spread out and no single family of cuts closes it neatly. On i080-214 my loop actually ran into the 2-hour time limit (7203 seconds) and still only reached 74.3%.

Separately, the cardinality cuts (my professor's Prop 1 / Prop 2 / CMI), when verified against the known optima, close i080-235, 305, 331 to 100% and 332 to 99.2%, and do nothing on instances with no exploitable cutset. That is the faithful reproduction of his method; the lift-and-project table above is the independent, optimum-free result.

7. The comparison that honestly reframes the whole project

Look at the last two columns of that table again. My cutting-plane loop takes anywhere from a couple of seconds to a **full two hours** (on i080-214). Gurobi's integer solver, given the same instances, solves every single one **to the exact optimum in well under 20 seconds**, most of them in under a second.

This was humbling but also, I think, the most important thing I learned. Modern solvers have advanced so much that these particular instances are simply not hard for them anymore. My professor even mentions this, that instances now have to be *pecially constructed* to be difficult, because the solvers got so good. My hand-built cutting planes are never going to beat Gurobi's decades of engineering at just *solving* the problem.

So what was the point? The point was never to beat the solver at solving. The value of this work is in **understanding the formulation and the bound**, why the LP sits where it does, why the cardinality cuts behave the way they do, why some cutsets are safe and some are not, and what it actually takes to raise the bound validly. That understanding is the real output. The solver gives you the answer in a second, but it does not tell you *why* the bound is 5888.5 or *why* a cardinality cut slides around at zero cost. Working that out by hand is what the project was really about.

8. What went wrong, what I could not do

I want to be straight about the limits of what I did.

The one thing I could not automate is the exact step my professor does by hand: picking the clean, "pass-through-free" cutsets by looking at the drawing. I tried several automatic ways to identify them (reachability, cost-based, heuristic witnesses) and each one either lets bad cutsets through or throws away good ones. My honest conclusion is that reliably identifying these cutsets seems to need a near-optimal solution in hand, which is close to already solving the problem and that is exactly why a human doing it by eye is genuinely useful. I think this is a real finding rather than a failure, but it is a limit.

Two instances (i080-214 and i080-344) don't fully close even with lift-and-project, and 214 hit the time limit. So the method is not a complete answer on the hardest cases.

The cardinality-cut results depend on the known optimum for verification. They are valid, but I should not oversell them as "solved with no information." The lift-and-project results are the ones that need nothing but the model.

9. Short summary

I set out to automate my professor's hand-drawn cardinality cuts for the Steiner Tree Problem. Along the way I rebuilt his EMCF-2 formulation faithfully, wrote code to find cutsets automatically, and implemented all three of his cut families. The big lesson was that the cuts rely on a "conservation" assumption that holds for his hand-picked cutsets but breaks under automatic search, which is why the hard part turned out to be *choosing* cutsets, not writing cuts. I got his cuts working honestly by checking each one against the published optimum, and I built an independent lift-and-project method that is valid by construction and closes 13 of 15 instances to 100% with no knowledge of the answer. Finally, comparing against Gurobi's own solver made it clear that the real value here is understanding the problem and its bound, not competing with a solver that already cracks these instances in seconds.